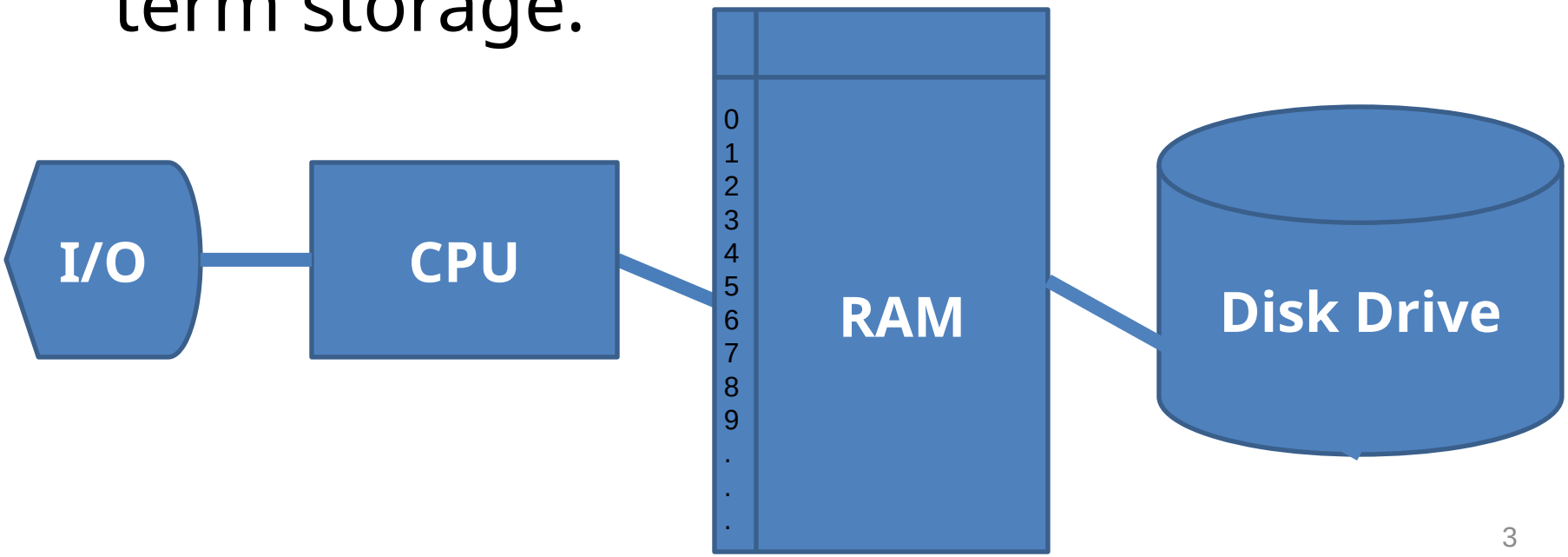
Operating Systems Security

Section 3.1: Operating Systems Concepts

A Computer Model

- An operating system has to deal with the fact that a computer is made up of a CPU, random access memory (RAM), input/output (I/O) devices, and long-term storage.



OS Concepts

- An **operating system (OS)** provides the interface between the users of a computer and that computer's hardware.
  - An operating system manages the ways applications access the resources in a computer, including its disk drives, CPU, main memory, input devices, output devices, and network interfaces.
  - An operating system manages multiple users.
  - An operating system manages multiple

Multitasking

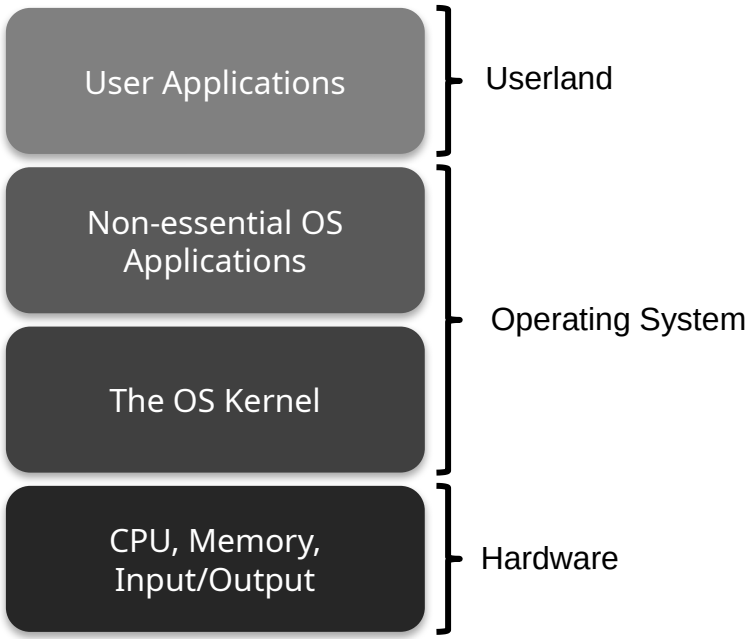
- Give each running program a “slice” of the CPU’s time.
- The CPU is running so fast that to any user it appears that the computer is running all the programs simultaneously.



Public domain image from http://commons.wikimedia.org/wiki/File:Chapters_meeting_2009_Liam_juggling.JPG

The Kernel

- The **kernel** is the core component of the operating system. It handles the management of low-level hardware resources, including memory, processors, and input/output (I/O) devices, such as a keyboard, mouse, or video display.



Input /Output

- **input/output devices**
  - Essential devices:
    - keyboard, mouse, video display card, and network card
  - optional devices:
    - scanner, Wi-Fi interface, video camera, USB ports, audio card, etc.
- **device driver:** encapsulates the details of how interaction with that device should be done.
  - **application programmer interface** (**API),** which the device drivers present to application programs
  - allows application programs to interact with devices at a fairly high level, while the operating system does the “heavy lifting” of performing the low-level interactions that make such devices actually work.

System Calls

- User applications don't communicate directly with low-level hardware components, and instead delegate such tasks to the kernel via **system calls**.
- System calls usually have root privilege that user codes do not have
  - Thus, user codes have to call system calls to access I/O or others

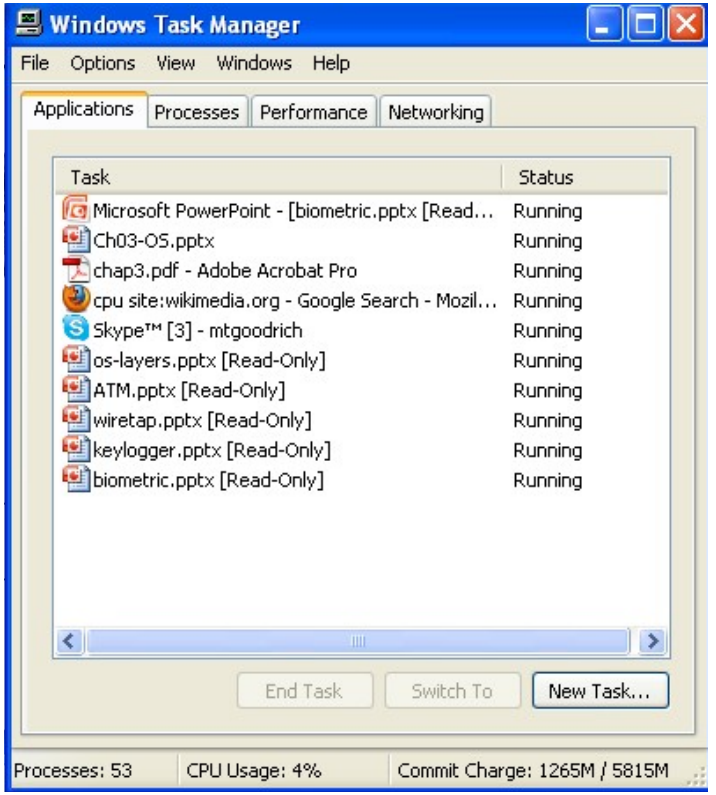
- Examples: file I/O (open, close, read, write) and running application programs (exec).

Difference between API and System Call?

- System calls are provided by OS Kernels.
- System calls are usually implemented via software interrupt
- System calls are the bridge between user code and OS kernel code
- User code can only use system calls to access privileged services or OS kernel data
- API are available code blocks that users can use directly
  - Facilitate programming, not essential

Processes

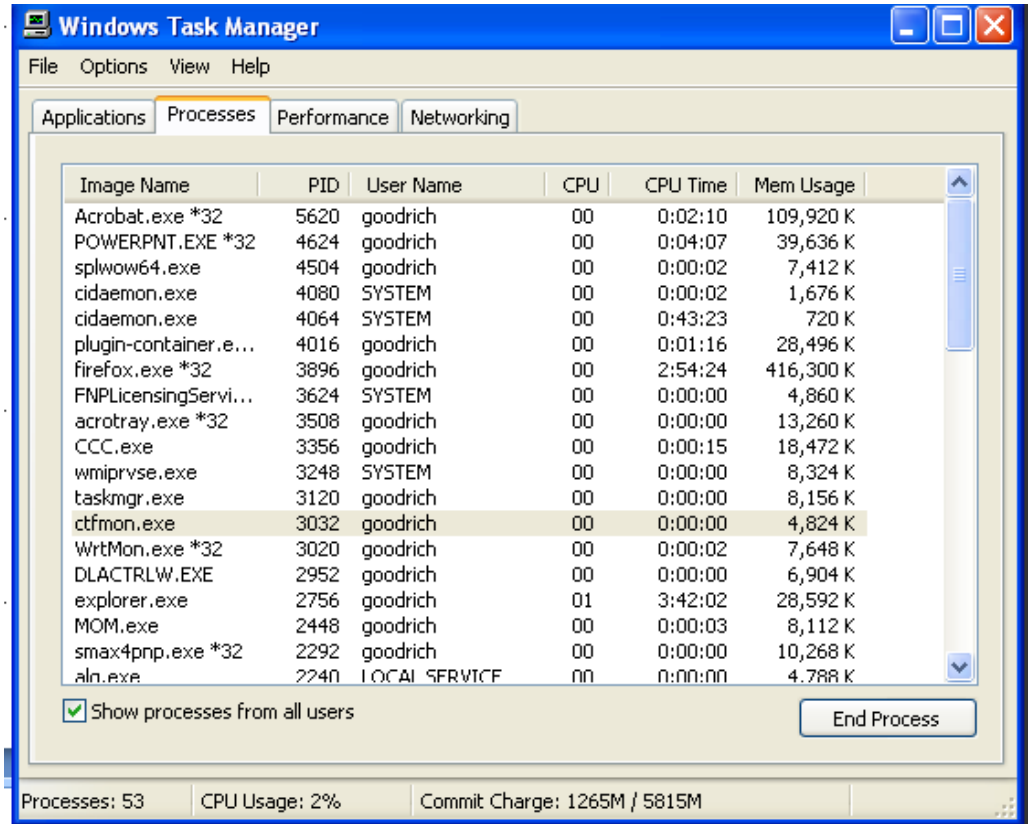
- A **process** is an instance of a program that is currently executing.
- The actual contents of all programs are initially stored in persistent storage, such as a hard drive.
- In order to be executed, a program must be loaded into random-access memory (RAM) and uniquely identified as a process.
- In this way, multiple copies of the same program can be run as different processes.



Process IDs

- Each process running on a given computer is identified by a unique nonnegative integer, called the **process ID (PID).**

- Given the PID for a process, we can then associate its CPU time, memory usage, user ID (UID), program name, etc.



Checking Process in Unix Machine

- PS command: listing processes running
  - "ps –aux" : list all users processes
    - May reveal private information of other users
- TOP command: display top CPU processes
- "top a": list top processes from all users

- PSTREE command:
  - shows running processes as a tree.

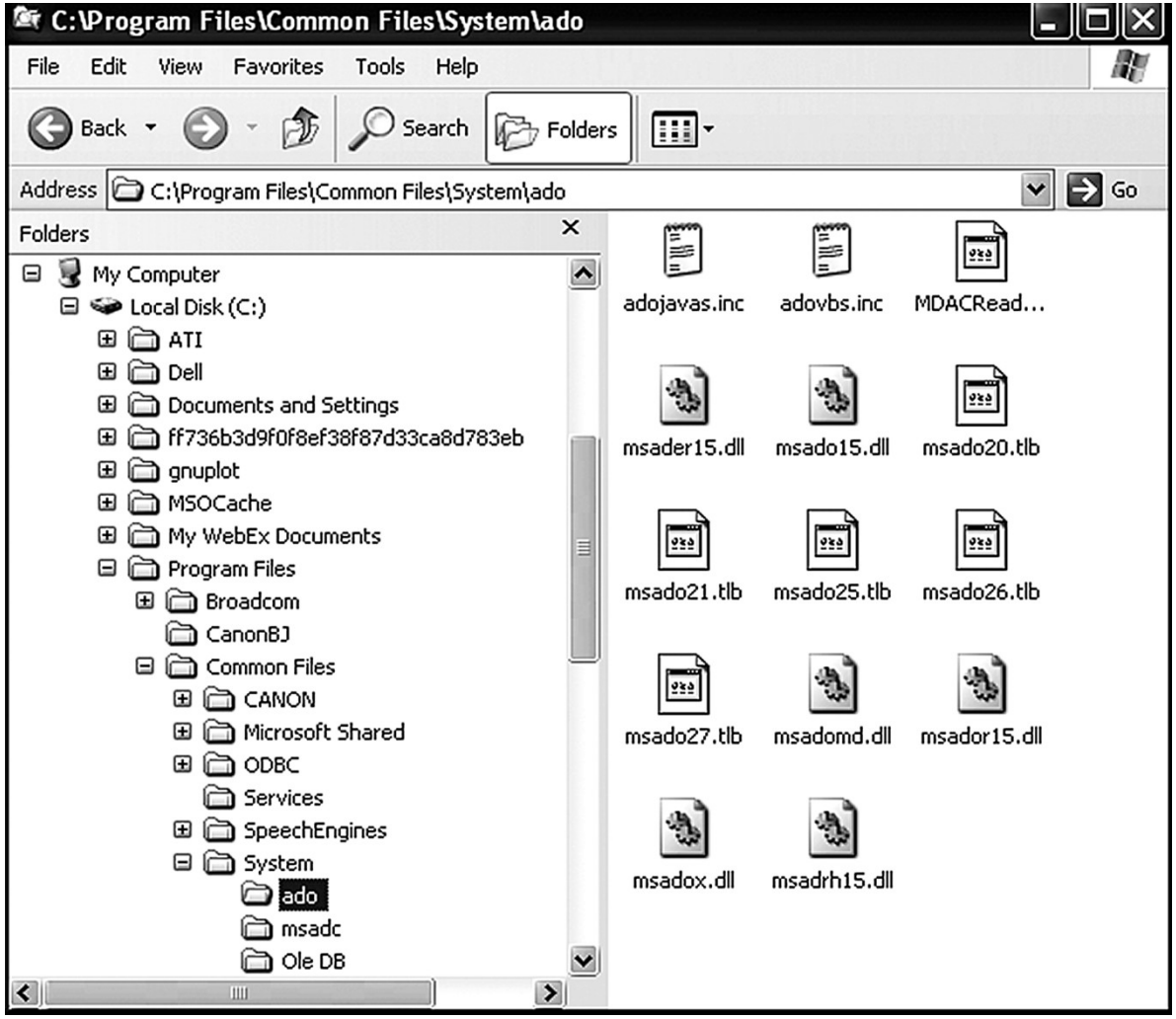
Process Privileges

- User ID (uid): identify the user associated with a process
- Group ID (gid): identify a group of users associated with a process
- Effective user ID (euid): determines the current privilege of the running process
  - Can set to be higher than uid in order to have permission to access OS kernel data or services

File Systems

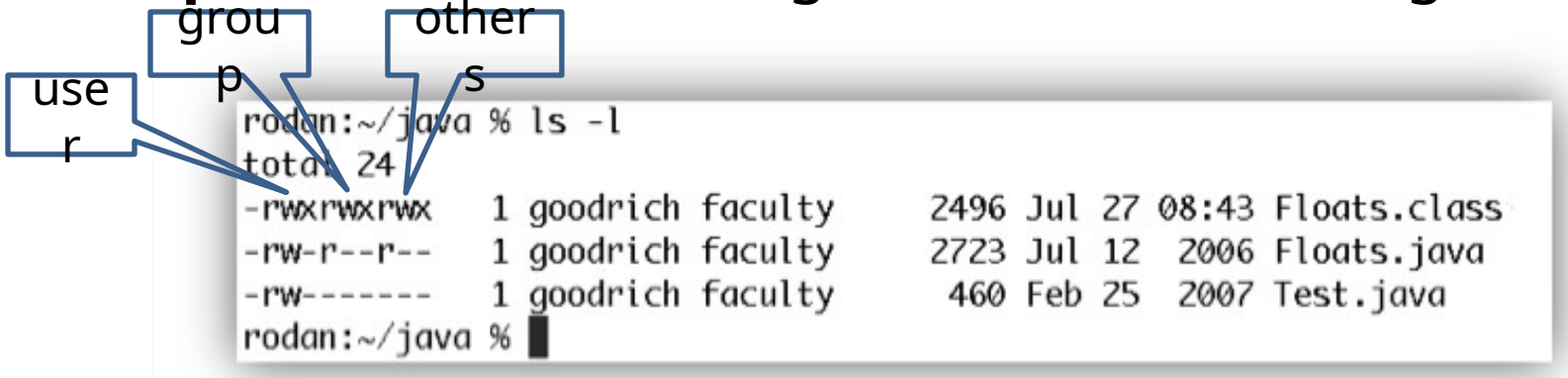
- A **filesystem** is an abstraction of how the external, nonvolatile memory of the computer is organized.
- Operating systems typically organize files hierarchically into **folders,** also called **directories.**
- Each folder may contain files and/or subfolders.
- Thus, a volume, or drive, consists of a collection of nested folders that form a **tree**.
- The topmost folder is the **root** of this tree and is also called the root folder.

File System Example



File Permissions

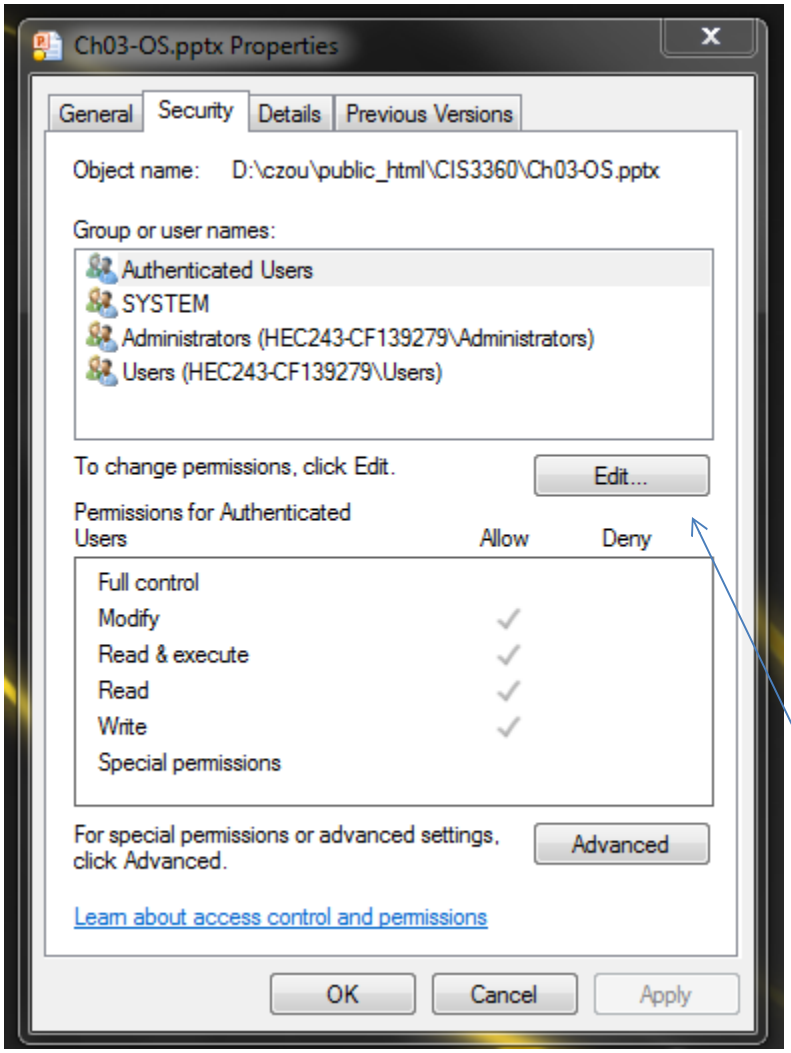
- File permissions are checked by the operating system to determine if a file is readable, writable, or executable by a user or group of users.
- In Unix-like OS's, a **file permission matrix** shows who is allowed to do what to the file.
  - Files have **owner permissions**, which show what the owner can do, and **group permissions**, which show what some group id can do, and **world permissions**, which give default access rights.

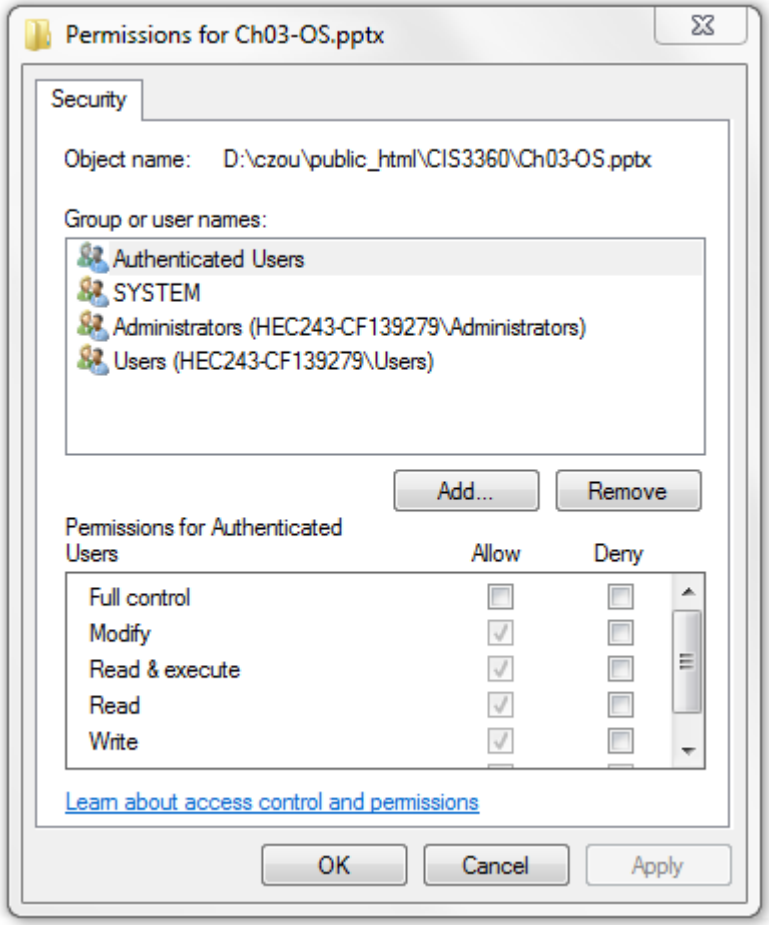


File Permissions

- Read bit: ‘r’
- Write bit: ‘w’
- Execute bit: ‘x’
- Folder permission:
  - ‘r’: list folder’s contents (such as ‘dir’, ‘ls’)
  - ‘w’: allow creation of new files in that folder
  - ‘x’ (search bit): allows you to enter the directory (e.g., cd command). However, you're not allowed to list its contents, unless you also have the read permissions to that directory.

File Permissions in Windows





You can edit permissions of a file/folder

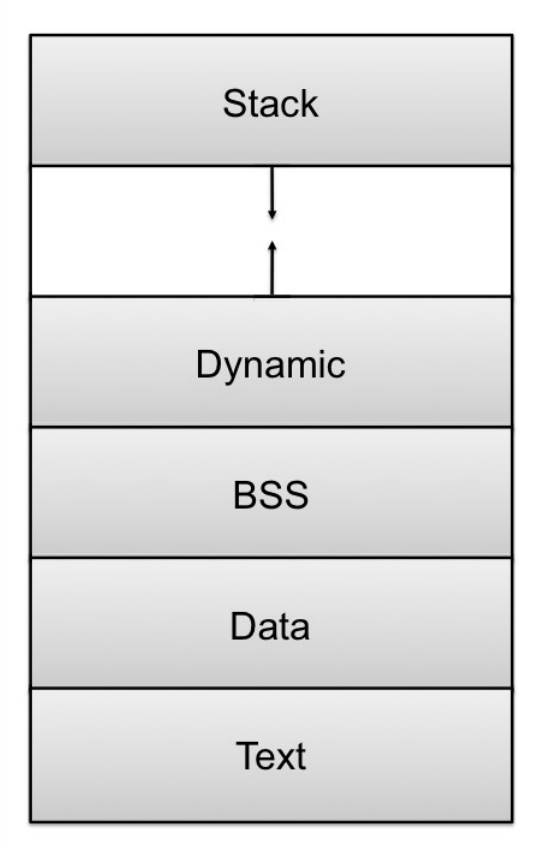
Memory Management

- The RAM memory of a computer is its **address space.**
- It contains both the code for the running program, its input data, and its working memory.
- For any running process, it is organized into different segments, which keep the different parts of the address space separate.
- As we will discuss, security concerns require that we never mix up these different segments.

Memory Organization

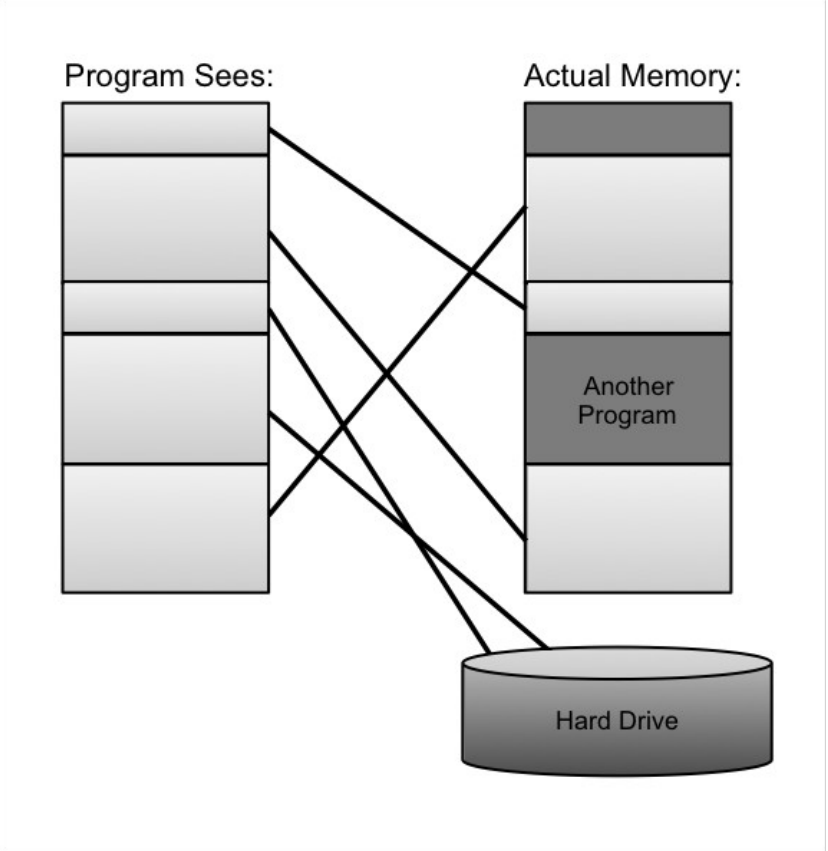
- **Text.** This segment contains the actual (binary) machine code of the program.
- **Data.** This segment contains static program variables that have been initialized in the program code.
- **BSS.** This segment, which is named for an antiquated acronym for block started by symbol, contains static variables that are uninitialized.
- **Heap.** This segment, which is also known as the dynamic segment, stores data generated during the execution of a process.
- **Stack.** This segment houses a stack data structure that grows downwards and is used for keeping track of the call structure of subroutines (e.g., methods in Java and functions in C) and their arguments.

Memory Layout



Virtual Memory

- There is generally not enough computer memory for the address spaces of all running processes.
- Nevertheless, the OS gives each running process the illusion that it has access to its complete (contiguous) address space.
- In reality, this view is **virtual**, in that the OS supports this view, but it is not really how the memory is organized.
- Instead, memory is divided into **pages**, and the OS keeps track of which ones are in memory and which ones are stored out to disk.



Page Faults

1. Process requests virtual address not in memory, causing a page fault.



Process

"read 0110101"

"Page fault, let me fix that."



Paging supervisor

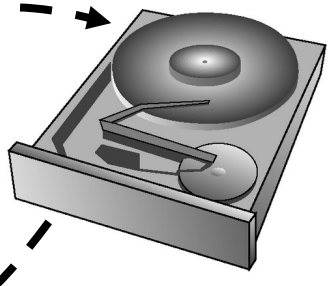
2. Paging supervisor pages out an old block of RAM memory.

Blocks in RAM memory:



old

new

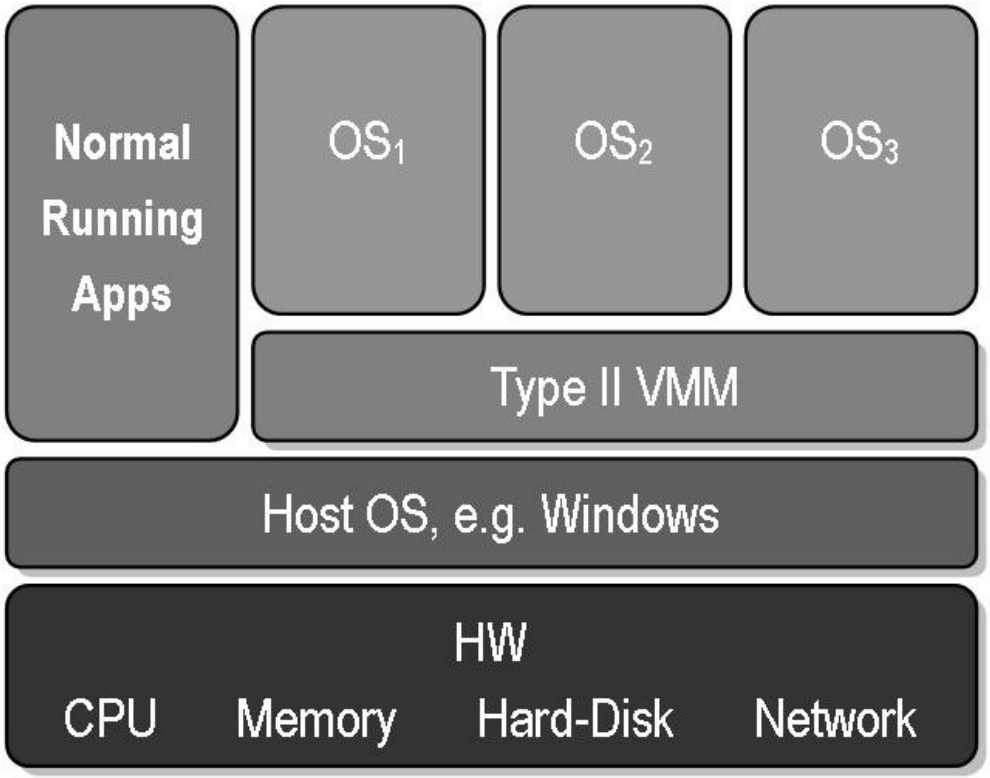


External disk

3. Paging supervisor locates requested block on the disk and brings it into RAM memory.

Virtual Machines

- **Virtual machine:** A view that an OS presents that a process is running on a specific architecture and OS, when really it is something else. E.g. a windows emulator on a Mac.
- **Benefits:**
  - **Hardware Efficienc**
  - **Portability**
  - **Security**
  - **Management**



Public domain image from http://commons.wikimedia.org/wiki/File:VMM-Type2.JPG

Two Types of Virtual Machine

- System Virtual Machine
  - Also called “hardware virtual machine’,
  - multiple OS environments can co-exist on the same computer
  - software that controls virtualization: "hypervisor" or "virtual machine monitor (VMM)"
- Process Virtual Machine
  - Also called ‘application virtual machine’
  - runs as a normal application inside a host OS
  - Its purpose is to provide a platform-independent physical machine emulation

Examples of VM

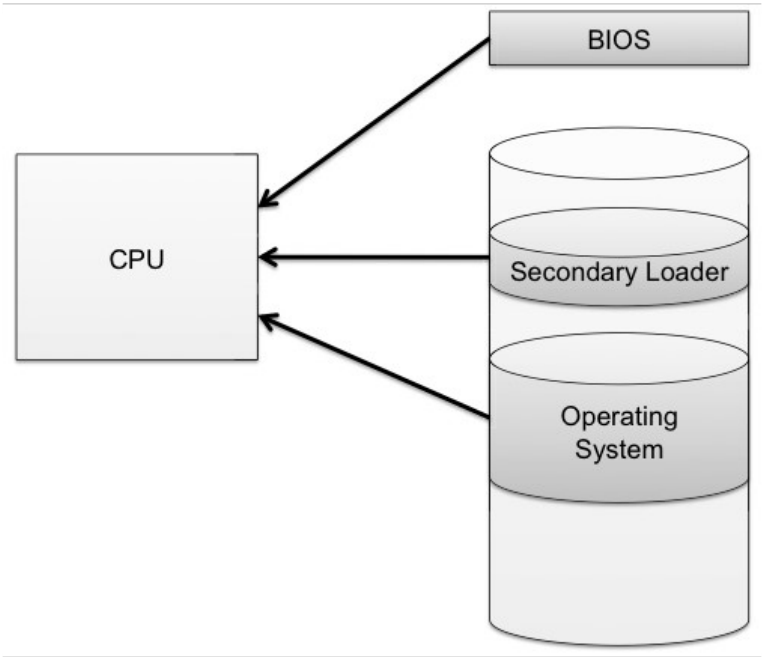
- Process VM
  - Java VM
  - .NET framework
  - Early version of Vmware
    - running as a normal program in Win/Linux to create a virtual machine that can be installed with any OSes.
- System VM
  - VMWare
  - Xen

Section 3.2: Operating Systems Security

The Boot Sequence

- When a computer is turned on, it first executes code stored in a firmware component known as the **BIOS (basic input/output system).**

- On modern systems, the BIOS loads into memory the **second-stage boot loader,** which handles loading the rest of the operating system into memory and then passes control of execution to the operating system.



Live CD attack

- A malicious user could potentially seize execution of a computer at several points in the boot process.

- **Live CD attack**: attackers use a bootable external media to boot up a computer in order to obtain hard drive data.
  - Original OS cannot protect data anymore.

BIOS Passwords

- **BIOS password:** that does not allow a second-stage boot loader to be executed without proper authentication.
  - Set up hard drive as the first boot device
  - **Prevent BIOS editing** without the password

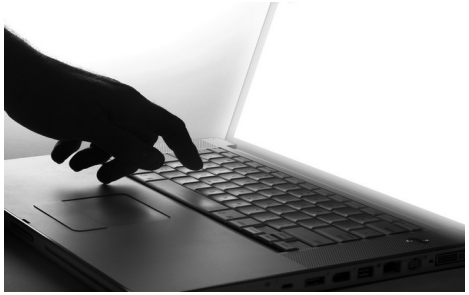
- Attackers can remove HD, install it in another computer to boot up
  - A better defense is to use **hard disk**

Hibernation

- Modern machines have the ability to go into a powered-off state known as **hibernation.**
  - OS stores the contents of machine's memory into a **hibernation file** (such as hiberfil.sys) on disk so the computer can be quickly restored later.
- But… without additional security precautions, hibernation exposes a machine to potentially invasive forensic investigation.



1. User closes a laptop computer, putting it into hibernation.

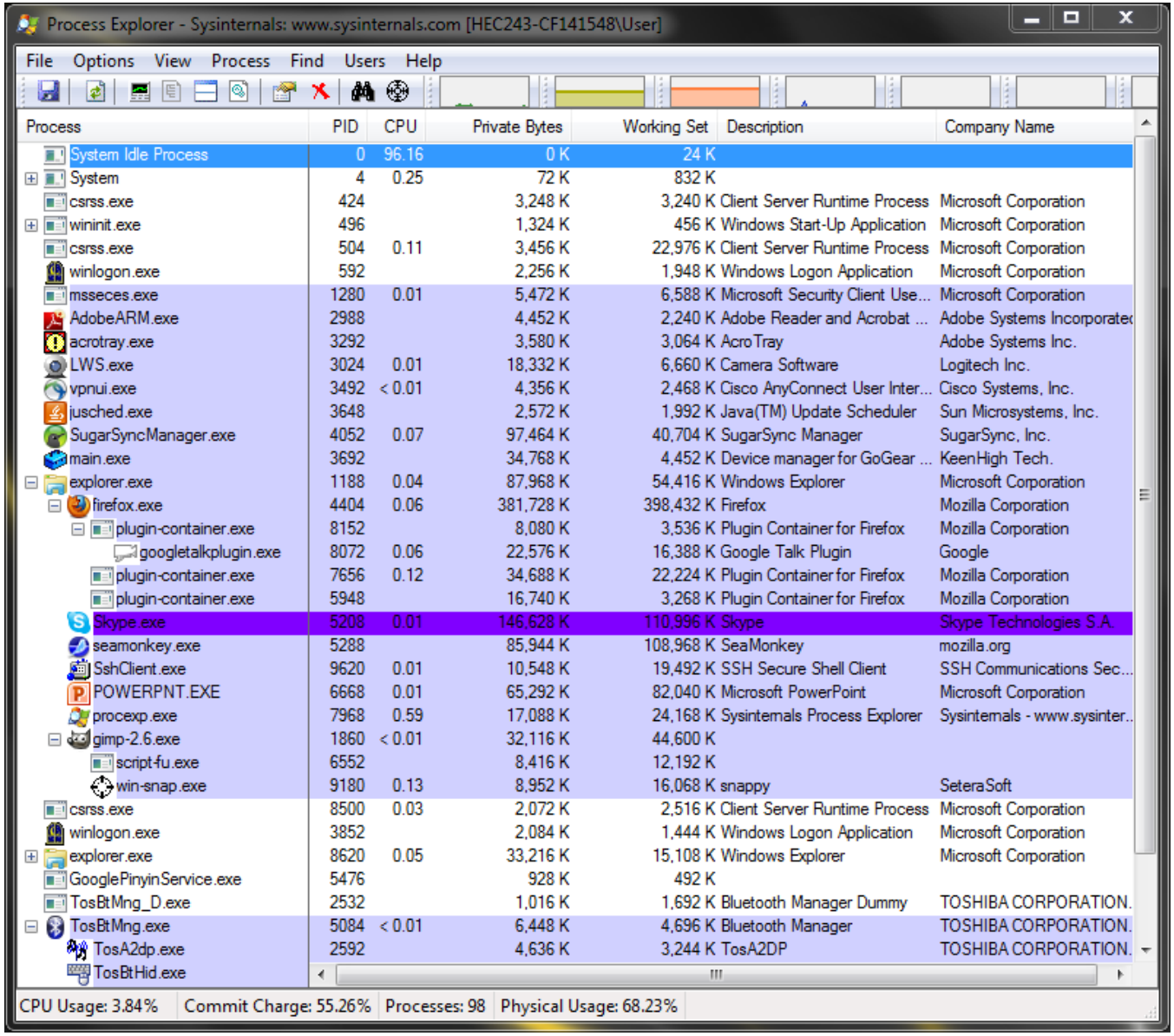


2. Attacker copies the hiberfil.sys file to discover any unencrypted passwords that were stored in memory when the computer was put into hibernation.

Event Logging

- Keeping track of what processes are running, what other machines have interacted with the system via the Internet,
- If the operating system has experienced any unexpected or suspicious behavior, it can often leave important clues not only for troubleshooting ordinary problems, but also for determining the cause of a security breach.

Process Explorer



Password Security

- Instead of storing passwords as plaintext, most OSes store the hash values in password files.
- **Dictionary attack:** with obtained password file, each word in a dictionary is hashed and the resulting value is compared with the hashed passwords stored in the password file.
- A dictionary of 500,000 “words” is often enough to discover most passwords.

Password Salt

- One way to make the dictionary attack more difficult to launch is to use **salt**.
- Associate a random number with each user id.
- OS compares the hash of (**password+salt**) with the stored hash of the (password+salt).

- Unix system password file:
  - /etc/passwd
  - Possible conjunction with /etc/shadow

How Password Salt Works

Without salt:

1. User types userid, X, and password, P.

2. System looks up H, the **stored hash** of X's password.

3. System tests whether h(P) = H.



Password file:

…
X: H
…

With salt:

1. User types userid, X, and password, P.

2. System looks up S and H, where S is the random salt for userid X and H is stored hash of S + X's password.

3. System tests whether **h(S||P)** = H.



Password file:

…
X: $E_k(S)$, H
…

How Salt Increases Search Space Size

- Assuming that an attacker cannot find the salt associated with a userid (e.g., salt is encrypted)
- then the search space for a dictionary attack on a salted password is of size $2^B * D$, where B is the number of bits of the random salt and D is the size of the list of words for the dictionary attack.
- For example, if a system uses a 32-bit salt for each userid and its users pick passwords in a 500,000 word dictionary, then the search space for attacking salted passwords would be

  $$\mathbf{2^{32} * 500{,}000} = 2{,}147{,}483{,}648{,}000{,}000$$

- Also, even if an attacker can find a salt password for a userid, he only learns one password.
  - Since different users' salts are different.